LAPORAN PRATIKUM PEMOGRAMAN BERORIENTASI OBJEK

"EKSPANSI PRODUK KEUANGAN (INHERITANCE)"



OLEH:

KHAULA LATHIFA FIRDAUSYI

2511531007

MATA KULIAH PEMOGRAMAN BERORIENTASI OBJEK
DOSEN PENGAMPU : NURFIAH, S.ST, M.KOM.
ASISTEN PRAKTIKUM ; EZZA ADDINI

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
2026

A. Tahapan Pelaksanaan Praktikum

1. Memastikan Akses Modifier di Rekening.java

Pada langkah ini, class Rekening diperiksa dan atribut saldo serta riwayatTransaksi diatur menggunakan modifier protected agar dapat diakses oleh subclass. Sementara itu, atribut pin dan nomorRekening tetap private untuk menjaga enkapsulasi.

Penambahan Kode Program Pada Rekening4.java:

```
package pekan4;

import java.util.ArrayList;

public class Rekening4 {

        // 1. Mengunci atribut dengan 'private'
        private String nomorRekening;
        private String namaPemilik;
        private String pin; // Data sensitif!

        // percobaan PIN salah & status blokir
        private int percobaanGagal = 0;
        private boolean terblokir = false;
        private static final int BATAS_PERCOBAAN = 3;

        // Implementasi Asosiasi (1-to-many)
        // Gunakan protected agar Subclass bisa mengaksesnya langsung
        protected double saldo;
        protected ArrayList<Transaksi3> riwayatTransaksi;

        // 2. Modifikasi Constructor untuk menerima PIN awal
        public Rekening4(String nomor, String nama, double saldoAwal, String pinAwal) {
                this.nomorRekening = nomor;
                this.namaPemilik = nama;
                this.saldo = saldoAwal;

                // Validasi PIN di dalam Constructor
                if (pinAwal.length() == 6) {
                        this.pin = pinAwal;
                } else {
                        System.out.println("Peringatan: PIN harus 6 digit! Menggunakan PIN
default 123456");
                        this.pin = "123456";
                }

                this.riwayatTransaksi = new ArrayList<>();
                System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat
dengan saldo " + saldo);
        }

        // 3. Getter untuk atribut yang diizinkan dibaca publik
        public String getNomorRekening() { return nomorRekening; }
        public String getNamaPemilik() { return namaPemilik; }

        // getter status blokir, hanya bisa dibaca dari luar, tidak bisa diubah langsung ---
```

```
        public boolean isTerblokir() { return terblokir; }

        // 4. Method Otentikasi Internal (Validasi Enkapsulasi)
        public boolean otentikasi(String inputPin) {
                if (terblokir) {
                        return false; // Sudah terblokir, otentikasi selalu gagal
                }

                if (this.pin.equals(inputPin)) {
                        percobaanGagal = 0; // reset hitungan kalau PIN benar
                        return true;
                } else {
                        percobaanGagal++;
                        if (percobaanGagal >= BATAS_PERCOBAAN) {
                                terblokir = true;
                        }
                        return false;
                }
        }
```

Atribut saldo dan riwayatTransaksi menggunakan protected sehingga subclass dapat mengakses keduanya secara langsung. Sebaliknya, nomorRekening, namaPemilik, dan pin tetap private sehingga akses langsung dari luar class tidak diperbolehkan. Constructor juga menerima PIN awal dan memeriksa panjangnya. Jika panjang PIN bukan 6 karakter, program menggunakan PIN default 123456. Bagian ini mendukung enkapsulasi sekaligus menjadi dasar yang digunakan oleh class turunan.

2. Membuat Subclass RekeningTabungan

Pada langkah ini dibuat file RekeningTabungan.java. RekeningTabungan menjadi subclass dari Rekening4 dan memiliki fitur tambahan berupa sukuBunga. Constructor subclass memanggil constructor superclass menggunakan super() sebelum mengisi atribut khusus tabungan. Modul menyebut produk tabungan memiliki fitur suku bunga bulanan.

Kode Program RekeningTabungan.java:

```
package pekan4;

public class RekeningTabungan extends Rekening4 {

        // Atribut spesifik yang hanya dimiliki oleh Tabungan
        private double sukuBunga;

        // Constructor Subclass
```

```
        public RekeningTabungan(String nomor, String nama, double saldoAwal, String pinAwal,
double sukuBunga) {
                //super() memanggil constructor kelas induk (Rekening). WAJIB berada di
baris pertama!
                super(nomor, nama, saldoAwal, pinAwal);
                this.sukuBunga = sukuBunga;
        }

        public void tambahBungaAkhirBulan() {
                // Menghitung bunga
                // Mengapa bisa mengakses saldo secara langsung dari class RekeningTabungan
?
                double nominalBunga = saldo * (sukuBunga / 100);
                saldo += nominalBunga;

                // Mencatat riwayat transaksi
                String idTrx = "TRX-B-" + System.currentTimeMillis();
                riwayatTransaksi.add(new Transaksi3(idTrx, "Bunga", nominalBunga));

                System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp" +
nominalBunga);
        }
}
```

Class RekeningTabungan menggunakan extends Rekening4 sehingga mewarisi atribut dan method dari superclass. Atribut sukuBunga dibuat private karena hanya digunakan sebagai data khusus tabungan. Method tambahBungaAkhirBulan() menghitung bunga berdasarkan saldo dan persentase suku bunga, kemudian menambahkannya ke saldo serta mencatat transaksi bunga ke riwayatTransaksi. Akses langsung ke saldo dan riwayatTransaksi dapat dilakukan karena kedua atribut tersebut protected.

3. Membuat Subclass RekeningGiro

Pada langkah ini dibuat file RekeningGiro.java. RekeningGiro merupakan subclass dari Rekening4 dan memiliki atribut tambahan berupa batasOverdraft sebagai limit pinjaman. Constructor memanggil constructor superclass menggunakan super().

Kode Program RekeningGiro.java:

```
package pekan4;

public class RekeningGiro extends Rekening4 {
        private double batasOverdraft;
```

```
        public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal,
double batasOverdraft) {
                // Memanggil inisialisasi dasar dari Superclass
                super(nomor, nama, saldoAwal, pinAwal);
                this.batasOverdraft = batasOverdraft;
        }

        // Getter khusu Giro
        public double getBatasOverdraft() {
                return batasOverdraft;
        }

        // (Catatan: Penarikan hingga limit overdraft akan kita selesaikan di Modul 5
}
```

Class RekeningGiro mewarisi Rekening4 menggunakan extends. Atribut batasOverdraft menyimpan batas overdraft khusus rekening giro. Constructor RekeningGiro menerima data dasar rekening dan batas overdraft, lalu memanggil super() untuk menjalankan constructor Rekening4. Method getBatasOverdraft() digunakan untuk membaca nilai batas overdraft dari luar class.

B. Latihan/Tugas

Tugas 1. Pembuatan Multi-Produk

Modifikasi case 1 pada Main.java agar setelah user memasukkan saldo awal dan PIN, program memberikan pilihan produk Tabungan Umum atau Giro Bisnis. Jika memilih Tabungan, user memasukkan sukuBunga dan program membuat objek RekeningTabungan. Jika memilih Giro, user memasukkan batasOverdraft dan program membuat objek RekeningGiro. Objek kemudian disimpan ke daftarRekening dengan konsep upcasting.

Kode Program Main4.java (bagian case 1):

```
case 1:
    System.out.print("Masukkan No Rekening: ");
    String no = input.nextLine();
    System.out.print("Masukkan Nama Pemilik: ");
    String nama = input.nextLine();
    System.out.print("Masukkan Saldo Awal: ");
```

```
    double saldo = input.nextDouble();
    input.nextLine();
    System.out.print("Buat PIN (6 digit angka): ");
    String  pinBaru = input.nextLine(); // buat PIN sebelum instansiasi
    System.out.println("\nProduk: ");
    System.out.println("1. Tabungan Umum");
    System.out.println("2. Giro Bisnis");
    System.out.print("Pilih produk: ");
    int produk = input.nextInt();
    input.nextLine();

    Rekening4 rekeningBaru = null; // tipe Superclass

    if (produk == 1) {
             System.out.print("Masukkan suku bunga (%): ");
             double sukuBunga = input.nextDouble();
             input.nextLine();
            // Upcasting: RekeningTabungan disimpan di variabel Rekening4
            rekeningBaru = new RekeningTabungan(no, nama, saldo, pinBaru, sukuBunga);
    } else if (produk == 2) {
            System.out.print("Masukkan batas overdraft: ");
            double batasOverdraft = input.nextDouble();
            input.nextLine();
            // Upcasting: RekeningGiro disimpan di variabel Rekening4
            rekeningBaru = new RekeningGiro(no, nama, saldo, pinBaru, batasOverdraft);
    } else {
            System.out.println("Pilihan produk tidak valid! Pembukaan rekening dibatalkan.");
            break;
    }
    daftarRekening.add(rekeningBaru); // Tambahkan ke ArrayList, tidak menimpa yang lama
    akunAktif= rekeningBaru; // Otomatis jadi akun aktif setelah dibuat
    break;
```

Variabel rekeningBaru menggunakan tipe Rekening4 sebagai superclass, tetapi objek yang dibuat dapat berupa RekeningTabungan atau RekeningGiro. Ini merupakan upcasting karena objek subclass disimpan dalam referensi superclass. Setelah objek berhasil dibuat, objek dimasukkan ke daftarRekening menggunakan add() dan dijadikan akunAktif. Dengan cara ini, satu ArrayList dapat menyimpan berbagai jenis rekening yang masih berada dalam hierarki Rekening4.

Tugas 2. Simulasi Waktu Akhir Bulan

Tambahkan menu 7 untuk simulasi akhir bulan khusus tabungan. Pada case 7, periksa apakah akunAktif merupakan objek RekeningTabungan menggunakan instanceof. Jika benar, lakukan downcasting kemudian panggil tambahBungaAkhirBulan(). Jika bukan tabungan, tampilkan pesan penolakan sesuai modul.

Kode Program Main4.java (case 7):

```
case 7:
    if (akunAktif == null) {
            System.out.println("Error: Anda belum membuka rekening!");
    } else if (akunAktif instanceof RekeningTabungan) {
            RekeningTabungan tab = (RekeningTabungan) akunAktif; // downcasting
            tab.tambahBungaAkhirBulan();
    } else {
            System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening
Tabungan.");
    }
    break;
```

Kode Program RekeningTabungan.java (method tambahBungaAkhirBulan):

```
public void tambahBungaAkhirBulan() {
            // Menghitung bunga
            // Mengapa bisa mengakses saldo secara langsung dari class RekeningTabungan ?
            double nominalBunga = saldo * (sukuBunga / 100);
            saldo += nominalBunga;

            // Mencatat riwayat transaksi
            String idTrx = "TRX-B-" + System.currentTimeMillis();
            riwayatTransaksi.add(new Transaksi3(idTrx, "Bunga", nominalBunga));

            System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp" +
nominalBunga);
    }
}
```

Operator instanceof digunakan terlebih dahulu untuk memastikan akunAktif benar-benar merupakan objek RekeningTabungan. Setelah pengecekan berhasil, dilakukan downcasting dari Rekening4 ke RekeningTabungan agar method khusus tambahBungaAkhirBulan() dapat dipanggil. Jika akun bukan RekeningTabungan, program menampilkan pesan bahwa fitur bunga akhir bulan hanya berlaku untuk rekening tabungan. Modul menekankan bahwa pengecekan instanceof diperlukan sebelum downcasting untuk menghindari ClassCastException.

C. Hasil Kode Program

```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi(Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Ringkasan Transaksi (challenge)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 123
Masukkan Nama Pemilik: abc
Masukkan Saldo Awal: 100000
Buat PIN (6 digit angka): 123456

Produk:
1. Tabungan Umum
2. Giro Bisnis
3. Rekening VIP (challenge)
Pilih produk: 1
Masukkan suku bunga (%): 5
Rekening atas nama abc berhasil dibuat dengan saldo 100000.0
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi(Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Ringkasan Transaksi (challenge)
0. Keluar
Pilih menu: 6
Masukkan PIN: 123456
=== MUTASI REKENING 123 ===
ID: TRX-B-1790953343931 | Jenis: Bunga | Nominal: Rp5000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi(Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Ringkasan Transaksi (challenge)
0. Keluar
Pilih menu: 4
--- INFO REKENING ---
No. Rekening : 123
Nama Pemilik : abc
Saldo Akhir  : Rp.105000.0
Status       : Aktif
---------------------
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi(Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Ringkasan Transaksi (challenge)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 123
Masukkan Nama Pemilik: abc
Masukkan Saldo Awal: 100000
Buat PIN (6 digit angka): 123456

Produk:
1. Tabungan Umum
2. Giro Bisnis
3. Rekening VIP (challenge)
Pilih produk: 2
Masukkan batas overdraft: 200000
Rekening atas nama abc berhasil dibuat dengan saldo 100000.0
```

D. Link Repository GitHub

https://github.com/khaulaathifa2/PrakPBO_C_2511531007